COMPREHENSIVE HACKATHON PROJECT SUBMISSION & LIFECYCLE REPORT

PROJECT: NOVA-X (FINAL MASTER REPORT)

Team Name: Nova X | **Repository:** GitHub - https://github.com/DulithThenuka/NOVA-X.git | **Submission Date:** June 20, 2026

---

1. Executive Problem Statement

Modern digital banking applications often suffer from fragmented navigation, broken routing workflows, and heavy cascade interferences between core styling layers. In high-speed, multi-developer environments like hackathons, parallel code deployment introduces additional friction. Sibling features frequently experience critical interface collisions, unmapped 404 target pathways, and system-level version control merge blocks.

Our team successfully engineered NOVA-X—a synchronized, high-performance FinTech platform that eliminates user interaction friction, handles state-driven banking tasks securely, maintains strict layout encapsulation, and integrates a localized relational database layer for persistent asset management.

2. Integrated Version Control Registry & Team Sprint Matrix

To ensure complete transparency and showcase highly structured agile collaboration within tight hackathon deadbounds, the official commit registry has been formalized below by grouping tasks into production-ready functional blocks:

2.1. Project Git History Logs

> **Commit Reference: 7e809d6e43ce1498e20b1f9a06a48bd490cde177 (HEAD -> main, origin/main)**
> Action: Conducted branch integration; successfully resolved downstream merge friction and pushed the stable version to the master stream.
>
> **Commit Reference: 494c84c3a019bb6765bd3ef248f69dd1c143481d**
> **Core Focus: Authentication UX Group**
> Action: Developed and refactored the structural Login User Interface layout.
>
> **Commit Reference: d9165a45f0e4182eee3d202e06176a2c376eb419**
> **Core Focus: Global Layout Engineering**
> Action: Built and deployed specialized Home Action and Logout buttons directly inside the global persistent Sidebar framework.

3. Product Architecture & Component Blueprint

The following matrix presents the finalized technical operational status of the platform modules built and validated during this hackathon development sprint:

Component / Module	Implementation File / Target	Operational Status	Technical Implementations & Architecture
API Home Page UI	app/api-home/page.tsx	**Operational**	Overhauled landing terminal layout connecting foundational action hubs safely.
Authentication Shell	/login (Core View)	**Operational**	Wireframed login modules and optimized view boundaries.
Global Navigation	Layout Sidebar Framework	**Operational**	Added reactive routing elements (Home / Logout actions).
Bank Transfer Wizard	app/bank-transfer/page.tsx	**Operational**	4-stage secure funds checkout workflow with client validation hooks.
Accounts Management	app/bank-accounts/page.tsx	**Fixed & Operational**	Wrapped route content in React Suspense boundary to fix prerendering checks.
Smart Spend Analytics	app/smart-spend/page.tsx	**Fixed & Operational**	Overhauled empty route file into a rich operational client view dashboard.
E-Statement Module	/e-statement	**Operational**	Viewport architecture setup for responsive data delivery.
Relational Database	PostgreSQL Instance (pg)	**Operational**	Integrated persistent client infrastructure with optimized schema nodes.

4.1. Defensive Top-Bar Core Interactions

- **Animated Expanding Utility Search:** Engineered an intuitive search filter that triggers smooth width expansions (w-0 to w-48/w-64) using Tailwind CSS transitions. It enables users to rapidly look up bank names against local structures.

- **Mutual Exclusion Dropdown Toggling:** Built strict event parameters ensuring that launching the Notification Tray Dropdown automatically collapses the active Search Input. This prevents overlapping layout crashes.
- **Sandbox Profile Avatar Isolation:** Designed the profile image component as an action-less static asset placeholder. This isolated structure ensures separate feature groups working on profile settings can inject dropdown paths or security links independently without blocking global code updates.

4.2. Local Relational Database Core Integration

To transition NOVA-X from a transient client state platform into a high-fidelity persistent FinTech terminal, a dedicated relational layer was integrated:

- **Database Architecture:** Engineered a persistent PostgreSQL relational engine using the native 'pg' driver and '@types/pg' structures to manage sensitive account metrics and real-time ledger entries securely.
- **Dependency Alignment:** Synchronized lockfile composition drift within package-lock.json to stabilize dependency compilation graphs for clean deployment environments.

4.3. Overhauled API Home Page & Ecosystem Wireframe Mapping

The central API landing gateway was re-engineered according to the structural ASCII-mapped layout schema, standardizing microservice routes across clean, boxed modular actions:

- **Ecosystem Core Title:** Deploys a multi-layered encapsulated wrapper housing 'Smart Spend: Nova Bank Integrated Financial Ecosystem'.
- **Triple Core-Action Hub Grid:** Features isolated routing targets for Account Wallets, Bank Transfers, and Bill Payments (Pay Bills) to compartmentalize navigation paths.
- **E-Statement Document Stream:** Provides direct sub-routing layout points dedicated to secure document extraction.
- **Dynamic Dashboard Launch:** Positions a unified dynamic bottom-anchored launcher ('Launch Smart Spend Dashboard') using interactive triggers to shift operational execution context safely.

4.4. State-Driven 4-Step Banking Wizard

The core funds transfer pipeline manages processing workflows through a checkout sequencer using distinct runtime states:

- **Form Entry Stage (form):** Collects institutional parameters. Employs regex boundaries to prevent invalid string submissions and secure non-negative amount constraints.
- **Review Stage (confirm):** Builds a transparent summary detailing input data alongside fixed processing charge metrics.
- **Success Stage (success):** Programmatically outputs a unique 8-digit random confirmation key string (confirmation) on successful operations.
- **Failure Fallback Stage (failure):** Intercepts data exceptions gracefully (e.g., balance threshold alerts) without terminating active application memory.

5. Overcoming Hackathon Challenges: Critical Bug Fixes & Refactoring

Time-constrained cross-branch merges create natural integration blocks. Our engineering team efficiently resolved the following structural anomalies during the development cycle:

5.1. Resolving the Accounts Module 404 Routing Bug

The Challenge: Initial workspace aggregation caused a fatal system break, throwing a 404 Page Not Found crash when a user attempted to navigate into the Accounts tab from the primary landing view.

The Resolution: Traced the link triggers and updated the root navigation file inside app/page.tsx, rerouting the href button parameter directly to point to the valid module endpoint directory: href="/accounts"

5.2. Next.js Suspense Boundary & Turbopack Core Fixes

- **Missing Suspense Boundary Solution:** Next.js build failed because useSearchParams() was used without a Suspense boundary while prerendering /bank-accounts. Wrapped the route content in React Suspense and moved the search-params logic into an inner AccountsContent component.
- **Turbopack Root Path Encapsulation:** Next.js warned that it inferred the wrong workspace root because another package-lock.json exists under user root paths. Set turbopack.root to process.cwd() inside next.config.ts so Next.js uses the NOVA-X project directory explicitly.
- **Biome Linting Validation:** Ran Biome formatting and safe assist fixes across multiple TypeScript/TSX files to normalize project formatting and sort import-orders cleanly.

```
// next.config.ts
turbopack: {
  root: process.cwd()
}
```

5.3. Final Verification Metrics

The fixed project compiles successfully. The following production checks completed with zero errors:

- **Linting:** npm run lint (Passes with zero diagnostics across all files under Biome standards)
- **Build Workflow:** npm run build (Next.js production build bundle finalized and compiled successfully)